

Lecture: Modelling and Shrinkage techniques

Actuarial Data Science - Open Learning Resource
Fei Huang, UNSW Sydney



Table of contents

- Statistical Machine Learning
- Shrinkage Techniques for Linear Models
- Shrinkage Techniques on Generalised Linear Models (GLM)
- Shrinkage Techniques on Other Models
- Using R
- Feature Selection



Recommended Reading

- Pre-requisite reading from An Introduction to Statistical Learning with Applications in R (ISLR): Chapters 2–7, 10
- The Elements of Statistical Learning (ESL): Chapter 4.4.4



Learning Objectives

1. Justify the importance of understanding of the business context when designing and implementing a data analytics project
2. Explain domain knowledge and discuss how it is used to interpret data and make feature selection
3. Explain the key iterative steps involved in building a model
4. Distinguish between different types of statistical machine learning, including regression and classification problems
5. Explain the iterative process of defining an appropriate target response variable in predictive modelling
6. Perform feature selection and transformation
7. Explain the purpose of dimension reduction and compare the advantages and disadvantages of different dimension reduction techniques



Learning Objectives (continued)

8. Explain and perform common techniques for splitting data into training, validation, and test sets
9. Understand shrinkage techniques
10. Apply shrinkage techniques in predictive modelling

In this lecture, we step back and examine the overall modelling landscape: what it means to build a model, how different statistical learning methods relate to one another, and why we sometimes deliberately *penalise* model complexity (shrinkage) to improve performance. The goal is to provide a conceptual map so that later techniques, such as GLMs, random forests, and boosting, fit into a coherent framework rather than appearing as isolated methods.



Statistical Machine Learning



Machine Learning vs. Statistics

- *Machine Learning* focuses on the design and development of algorithms that allow computers (machines) to improve their performance over time based on data.
- *Statistics* classically often requires or assumes knowledge of the data-generating process (modelling), including the probability distribution from which the data are drawn.
- The distinction between statistics and machine learning has blurred over the past few decades
- *Actuarial data analytics* benefit from both areas



Different Types of Learning

- **Supervised Learning**

- Given pairs of input data (predictors) and targets (labels)
- Learn a mapping from inputs to targets (training)
- Goal: use the learned mapping to predict targets for new input data
- Includes regression and classification

- **Unsupervised Learning**

- Only input data are given $x_i, i = 1, \dots, n$, with no responses (targets/labels) y_i
- Goal: understand relationships between variables –
- Often used for data exploration (e.g. cluster analysis)



Different Types of Learning (continued)

- **Semi-supervised Learning**
 - Labelling data can be expensive or difficult
 - Some observations have associated responses, while others do not
- **Reinforcement Learning**
 - Agents learn to maximise rewards through interaction with an environment
 - e.g. used in AlphaGo



The 5 Core Activities of Data Analysis

1. Stating and refining the question
2. Exploring the data
3. **Building formal statistical models**
4. Interpreting the results
5. Communicating the results



The Iterative Modelling Process

1. Business understanding
2. Data understanding and preparation
3. Modelling
4. Evaluation
5. Communication
6. Deployment
 - Application of a model to make predictions on new data
 - e.g. generating reports or implementing a repeatable data science process



Defining Your Target Variable

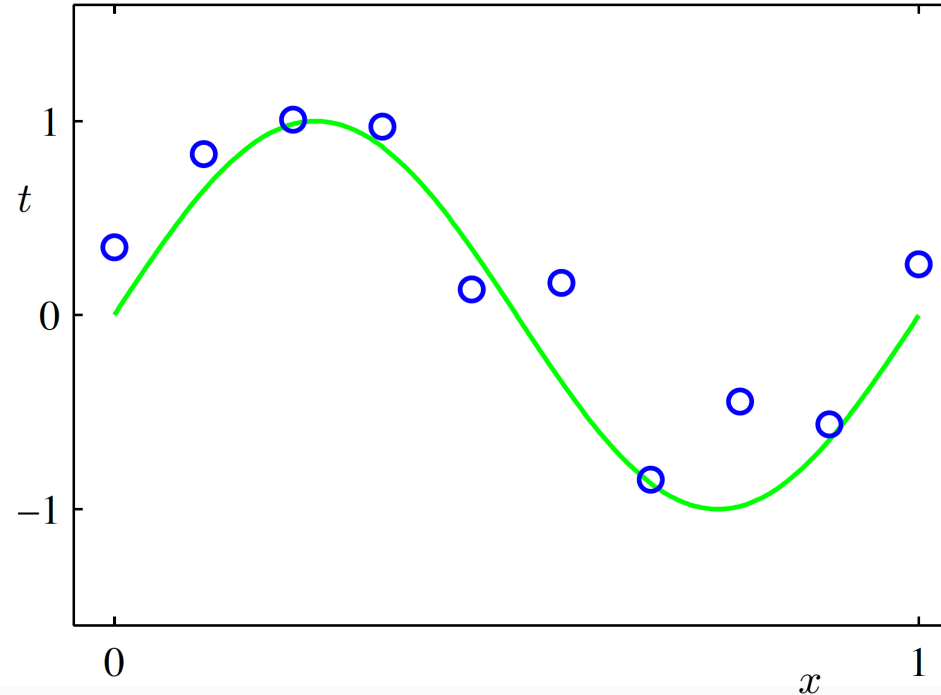
- Use business and domain knowledge
- Think carefully about the question
- Follow the iterative data analysis process to refine it
- Multiple options may be possible — which one is most appropriate?



Modelling Example: Polynomial Curve Fitting

- We simulate artificial training data from the function

$$\sin(2\pi x) + \text{Gaussian random noise}, \quad x \in [0, 1]$$



Data generation (Source: Bishop and Nasrabadi (2006), Figure 1.2)



Training Data

- Sample size: $N = 10$
- Input variable (predictor): $x \equiv (x_1, \dots, x_N)^T$
- Target variable (response): $y \equiv (y_1, \dots, y_N)^T$
- x is generated by choosing x_n ($n = 1, \dots, N$) uniformly from $[0, 1]$ (R code: `runif()`)
- y is generated by $\sin(2\pi x) + \text{Gaussian random noise}$ (R code: `sin()`, `rnorm()`)
- **Goal:** predict the target value $\hat{y}$ for a new input $\hat{x}$ (i.e. achieve good generalisation)



Model Specification

- Polynomial Regression

$$f(x, \mathbf{w}) = w_0 + w_1x + \cdots + w_Mx^M = \sum_{j=0}^M w_jx^j$$

- M is the order of the polynomial
- $f(x, \mathbf{w})$ is a nonlinear function of x
- $f(x, \mathbf{w})$ is a linear function of the parameters $\mathbf{w}$
- How can we find suitable parameters $\mathbf{w} = (w_0, w_1, \dots, w_M)^T$?

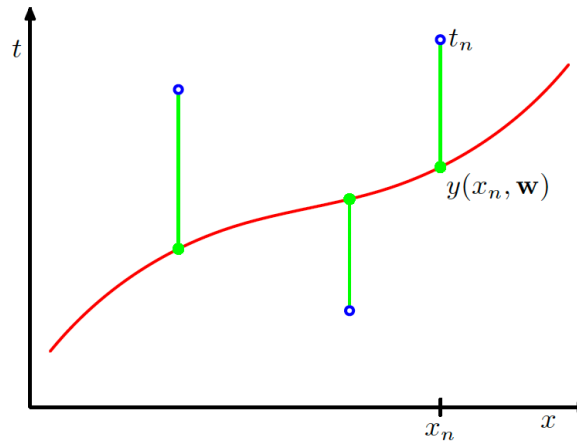


Performance Measure

- Minimise an error function, such as the sum of squared errors

$$\text{Err}(\mathbf{w}) = \sum_{n=1}^N (f(x_n, \mathbf{w}) - y_n)^2$$

- A unique closed-form solution $\mathbf{w}^*$ that minimises $\text{Err}(\mathbf{w})$ can be obtained
- How should we choose the order M of the polynomial?



Geometric interpretation of the sum-of-squares error function (Source: Bishop and Nasrabadi (2006), Figure. 1.3)



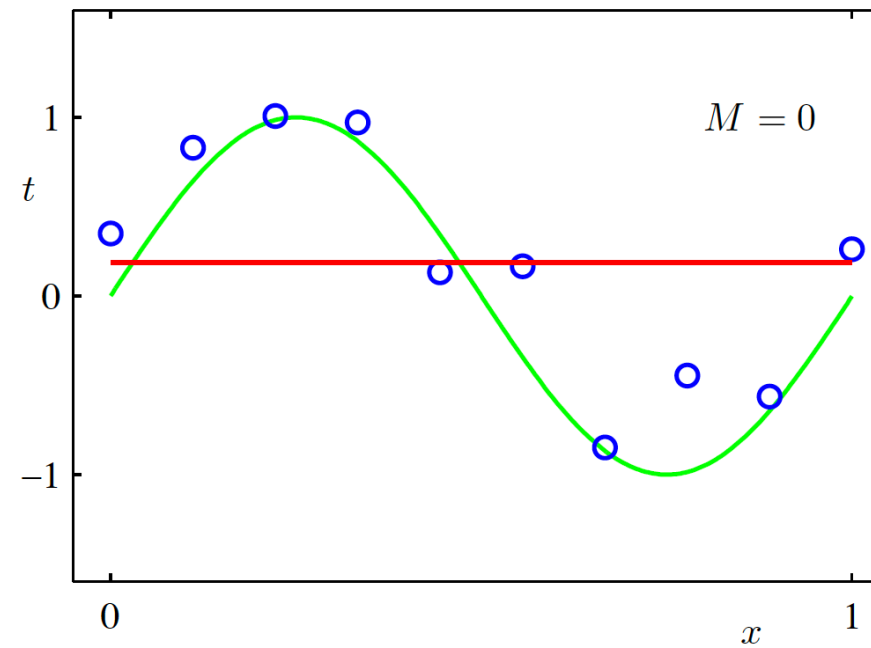
Model Selection

- Fit models with different polynomial orders:
 - $M = 0, 1, 3, 9$
- Compare how model complexity affects the fit



Constant Model ($M = 0$)

$$\begin{aligned} f(x, \mathbf{w}) &= \sum_{m=0}^M w_m x^m \Big|_{M=0} \\ &= w_0 \end{aligned}$$

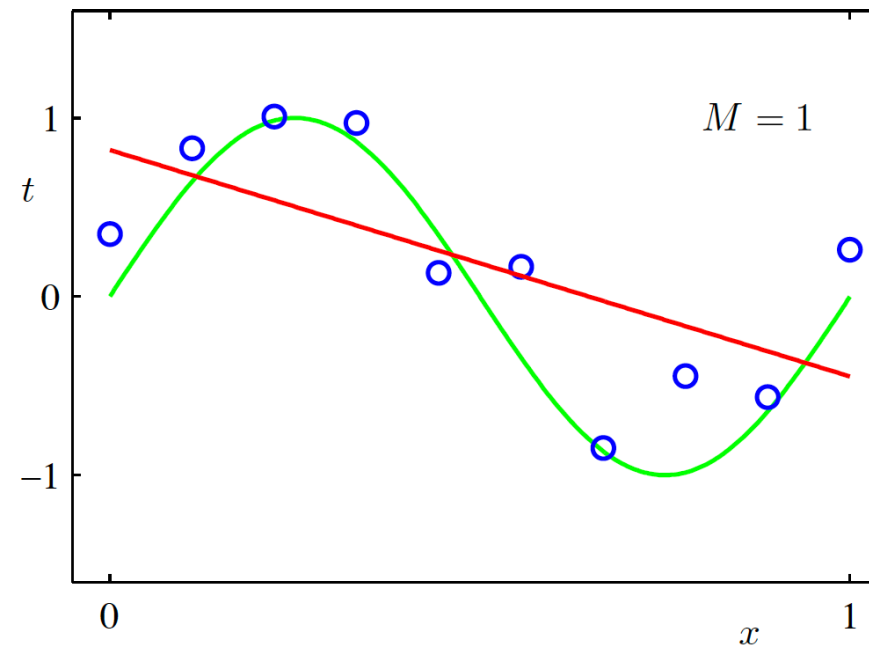


Fitted polynomial ($M = 0$) (Source: Bishop and Nasrabadi (2006), Figure. 1.4)



Linear Model ($M = 1$)

$$\begin{aligned} f(x, \mathbf{w}) &= \sum_{m=0}^M w_m x^m \Big|_{M=1} \\ &= w_0 + w_1 x \end{aligned}$$



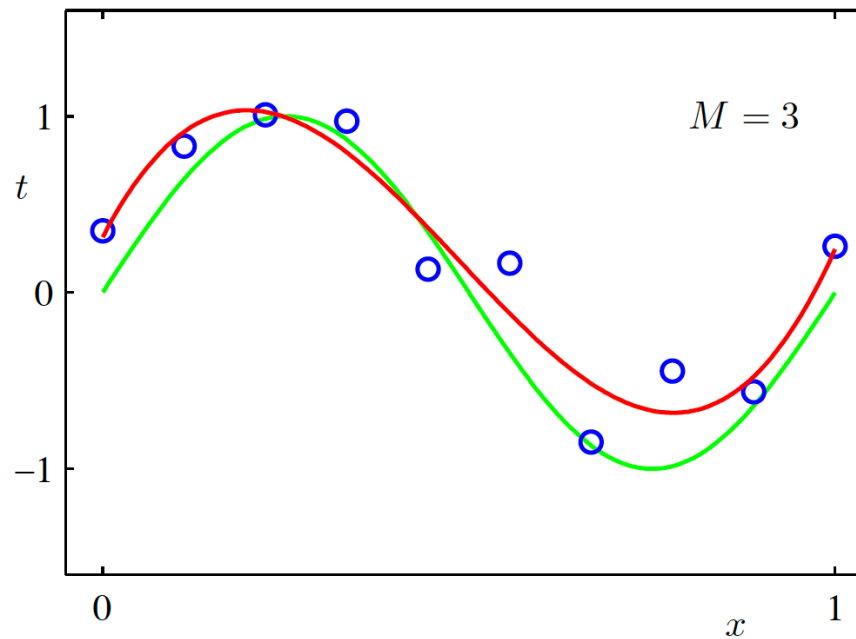
Fitted polynomial ($M = 1$) (Source: Bishop and Nasrabadi (2006), Figure. 1.4)



Cubic Polynomial Model ($M = 3$)

$$f(x, \mathbf{w}) = \sum_{m=0}^M w_m x^m \Big|_{M=3}$$

$$= w_0 + w_1 x + w_2 x^2 + w_3 x^3$$



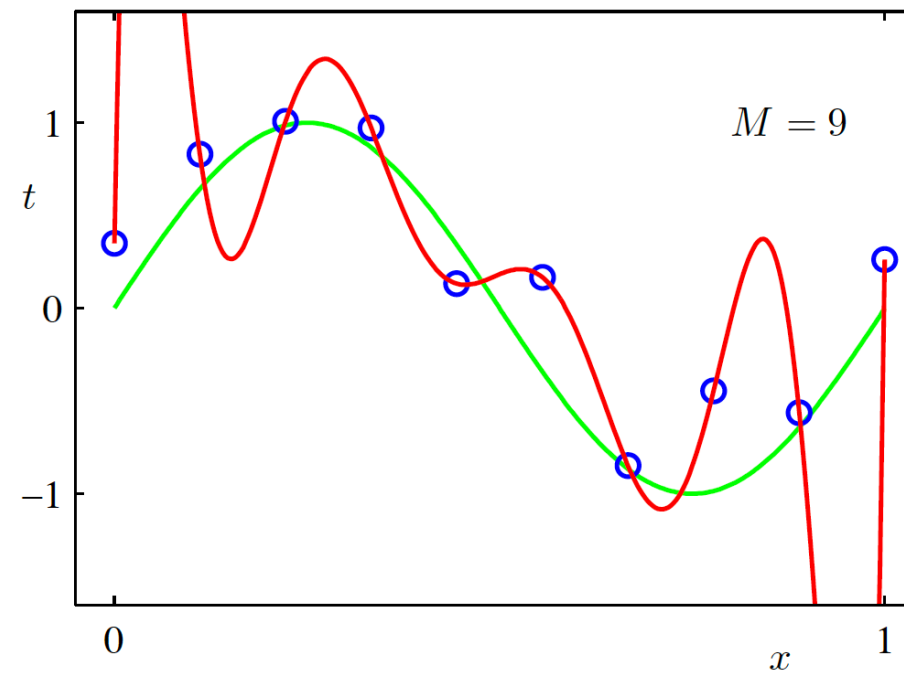
Fitted polynomial ($M = 3$) (Source: Bishop and Nasrabadi (2006), Figure. 1.4)



High-Degree Polynomial Model ($M = 9$)

$$f(x, \mathbf{w}) = \sum_{m=0}^M w_m x^m \Big|_{M=9}$$

$$= w_0 + w_1 x + \cdots + w_9 x^9$$



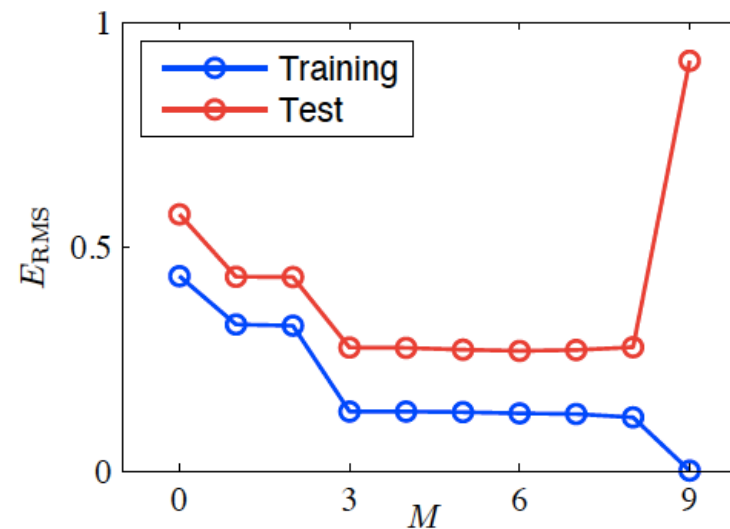
Fitted polynomial ($M = 9$) (Source: Bishop and Nasrabadi (2006), Figure. 1.4)



Testing the Fitted Model

- Train the model and obtain $\mathbf{w}^*$
- Generate 100 new observations as a test set
- Root-mean-square (RMS) error

$$Err_{RMS} = \sqrt{2Err(\mathbf{w}^*)/N}$$



Training and test error vs model complexity (Source: Bishop and Nasrabadi (2006), Figure. 1.5)



Parameters of the Fitted Model

	M = 0	M = 1	M = 3	M = 9
w_0^*	0.19	0.82	0.31	0.35
w_1^*		-1.27	7.99	232.37
w_2^*			-25.43	-5321.83
w_3^*			17.37	48568.31
w_4^*				-231639.30
w_5^*				640042.26
w_6^*				-1061800.52
w_7^*				1042400.18
w_8^*				-557682.99
w_9^*				125201.43

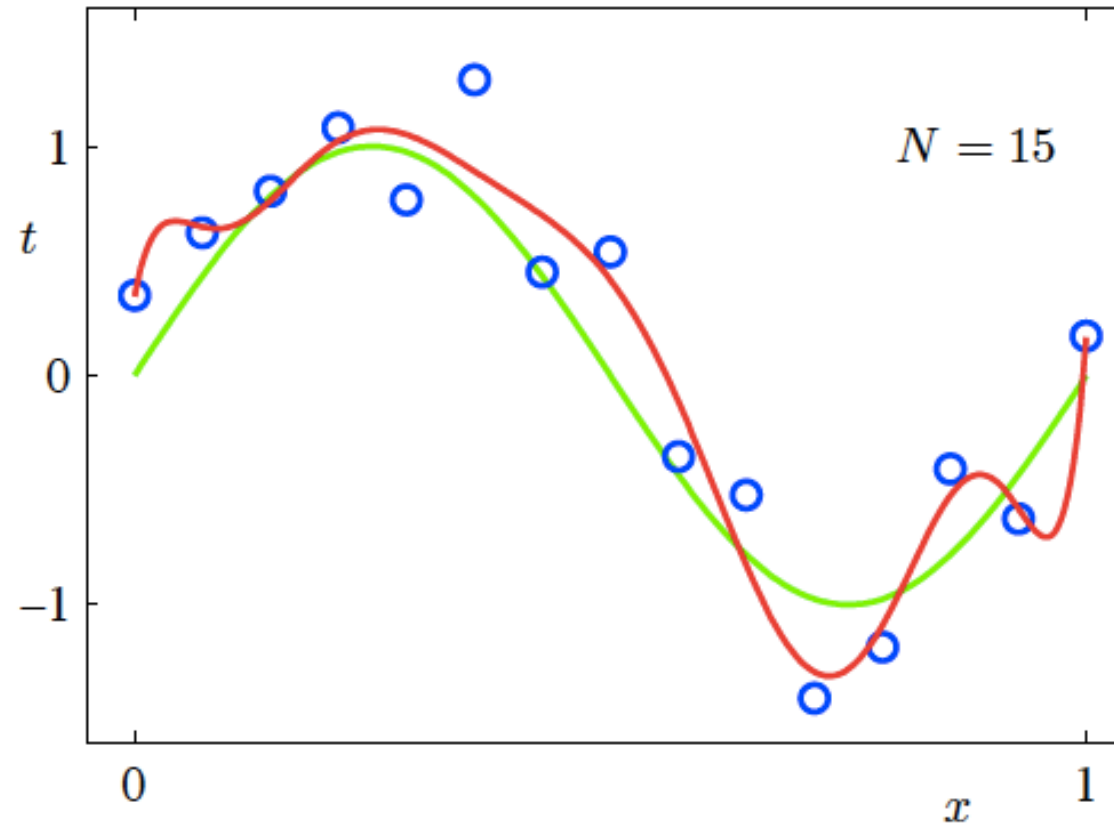
Table: Coefficients w^* for polynomials of various order.

Fitted model parameters for different polynomial orders (Source: Bishop and Nasrabadi (2006), Table 1.1)



Effect of Training Data Size

- $N = 15, M = 9$

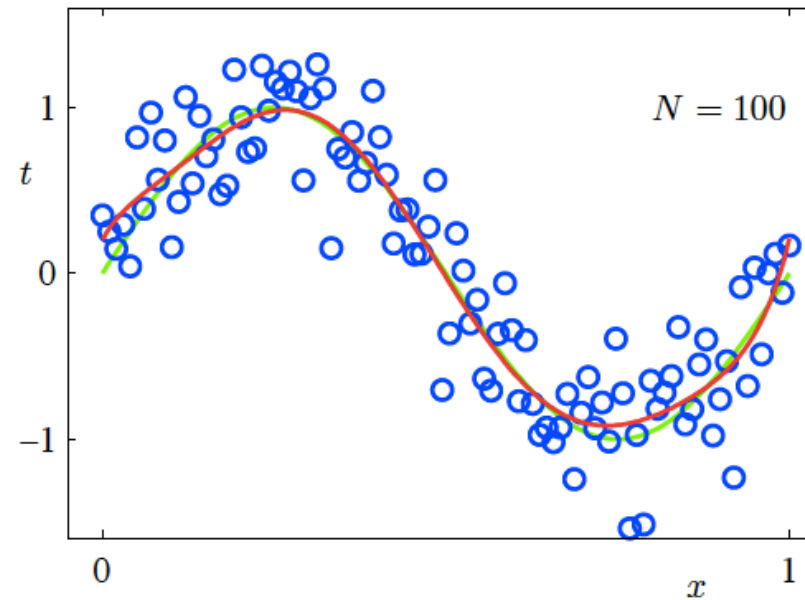


Effect of training set size on model fitting ($N = 15, M = 9$) (Source: Bishop and Nasrabadi (2006), Figure. 1.6)



Increasing the Training Data Size

- $N = 100, M = 9$
- Increasing the size of the dataset reduces overfitting
- The number of data points should be no less than some multiple (say 5 or 10) of the number of parameters in the model



Effect of training set size on model fitting ($N = 100, M = 9$) (Source: Bishop and Nasrabadi (2006), Figure. 1.6)



How?

- How can we control overfitting?
- How can we control model complexity?



Shrinkage Techniques

- Add a penalty (regularisation) term to the error function in order to control overfitting

$$\tilde{\text{Err}}(\boldsymbol{w}) = \text{Err}_D(\boldsymbol{w}) + \lambda \text{Err}_W(\boldsymbol{w})$$

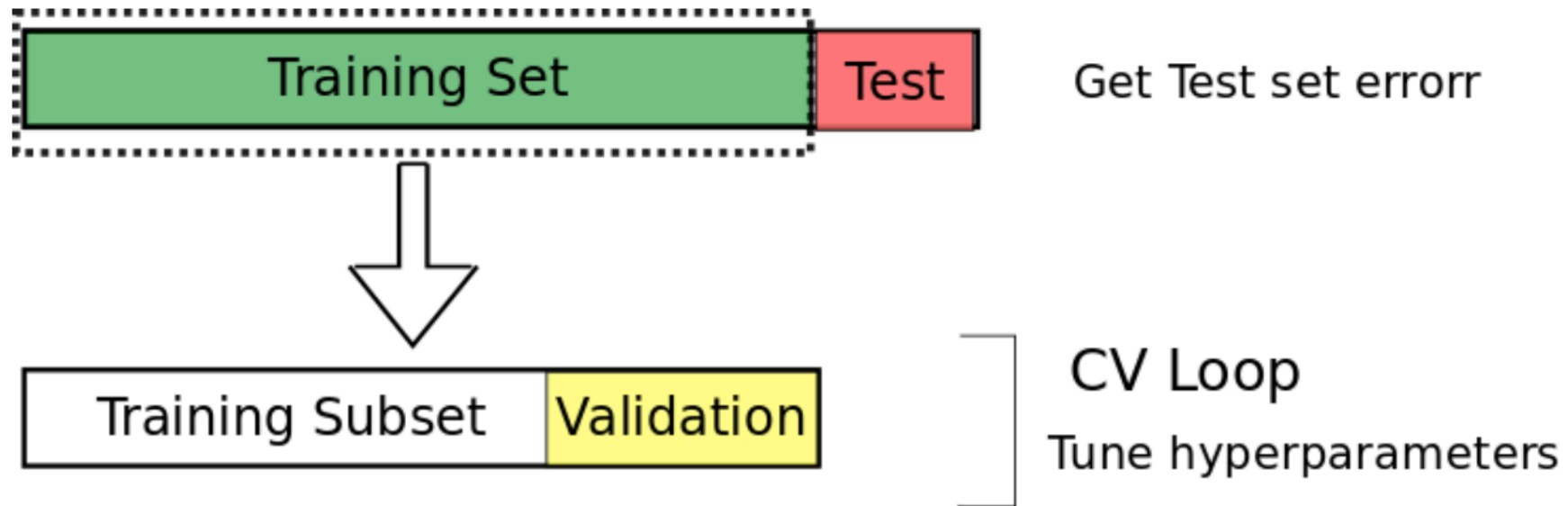
- λ is the regularisation coefficient that controls the relative importance of the *data-dependent error* $\text{Err}_D(\boldsymbol{w})$ and the *regularisation term* $\text{Err}_W(\boldsymbol{w})$
- How do we choose λ ?

Review: *An Introduction to Statistical Learning* (James et al. 2013), Chapter 6.2



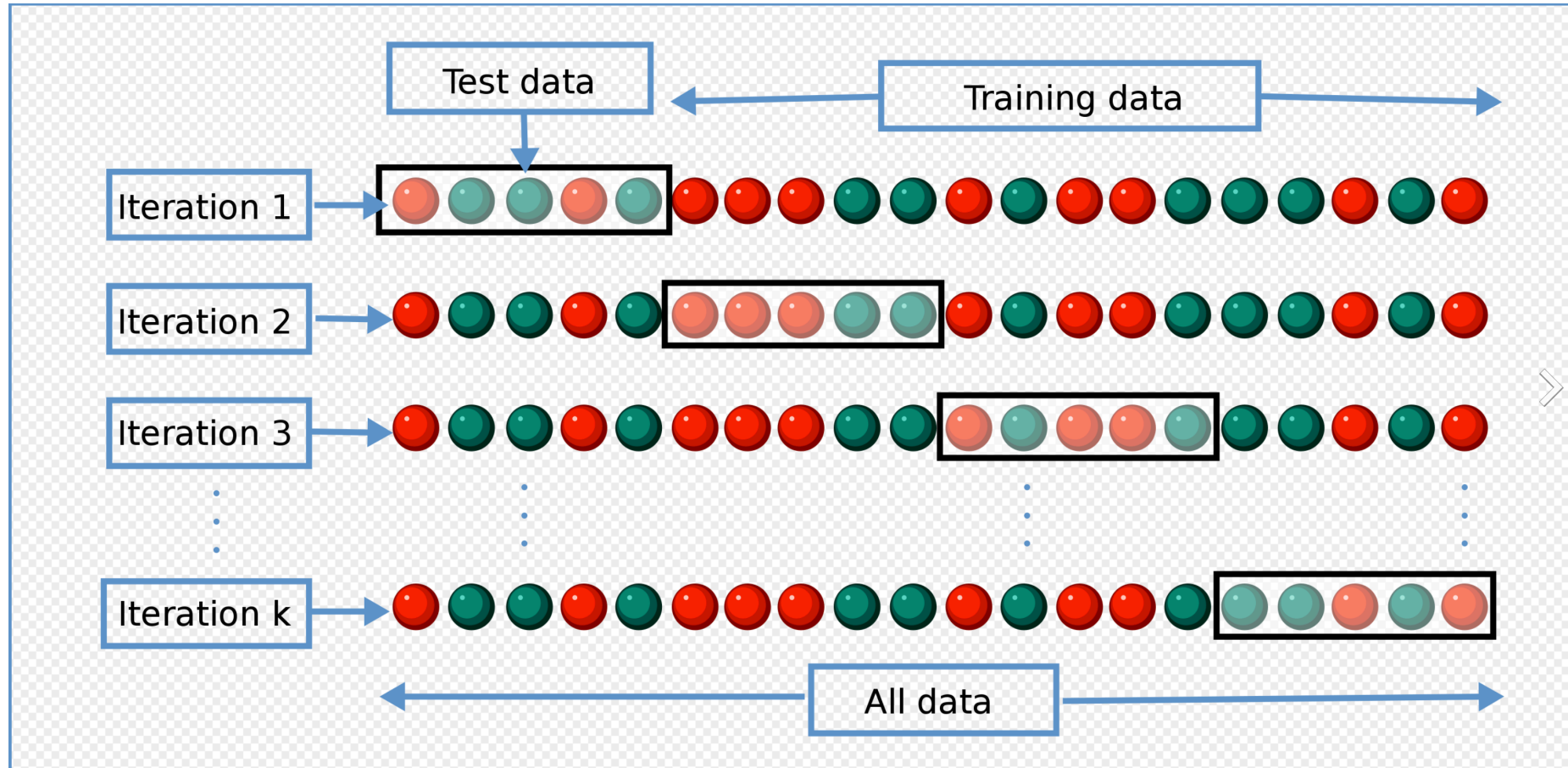
Data Splitting and Cross Validation

- Split the data into training, validation and testing sets



Data splitting (Source: Cochrane (2018))

- Cross validation



Cross-validation (Source: Wikimedia Commons contributors (2016))

Review: *An Introduction to Statistical Learning* (James et al. 2013), Chapter 5.1

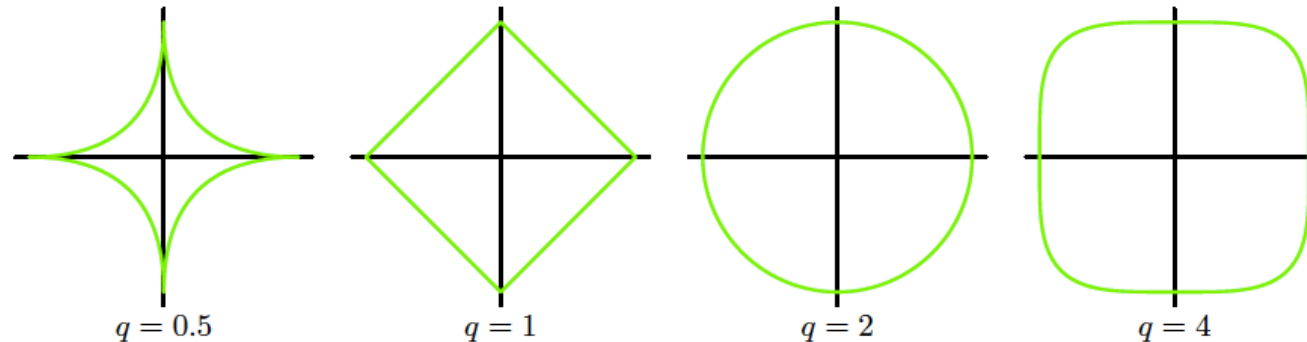


A general penalty/regulariser

- Regularization term: $\text{Err}_W(\mathbf{w}) = \sum_{j=1}^M |w_j|^q = \|\mathbf{w}\|^q$
- The regularised error is

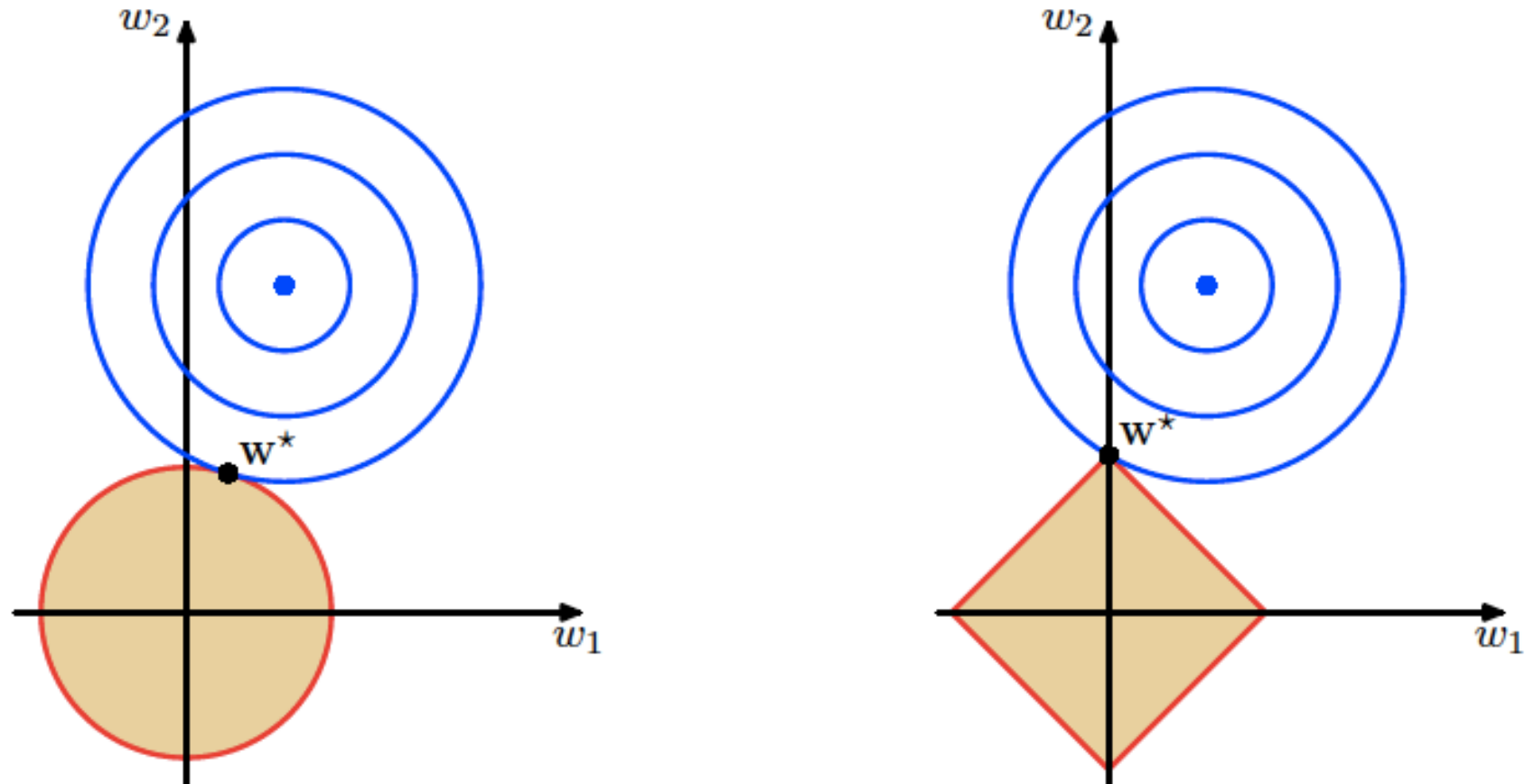
$$\tilde{\text{Err}}(\mathbf{w}) = \sum_{n=1}^N [f(x_n, \mathbf{w}) - t_n]^2 + \lambda \|\mathbf{w}\|^q$$

- $q = 1$: L1 regularisation (lasso)
- $q = 2$: L2 regularisation (ridge)



Geometric interpretation of L1 and L2 regularisation (Source: Bishop and Nasrabadi (2006), Figure. 3.3)

Geometric Comparison of Ridge and Lasso



Geometric comparison of ridge and lasso regularisation (Source: Bishop and Nasrabadi (2006), Figure. 3.4)

Modelling Example: Ridge (L2) Regularisation

- Add the ridge regulariser to the error function to discourage the coefficients from reaching large values and control model complexity

$$\tilde{\text{Err}}(\mathbf{w}) = \sum_{n=1}^N [f(x_n, \mathbf{w}) - y_n]^2 + \lambda \|\mathbf{w}\|^2$$

- Squared norm of the parameter vector $\mathbf{w}$:

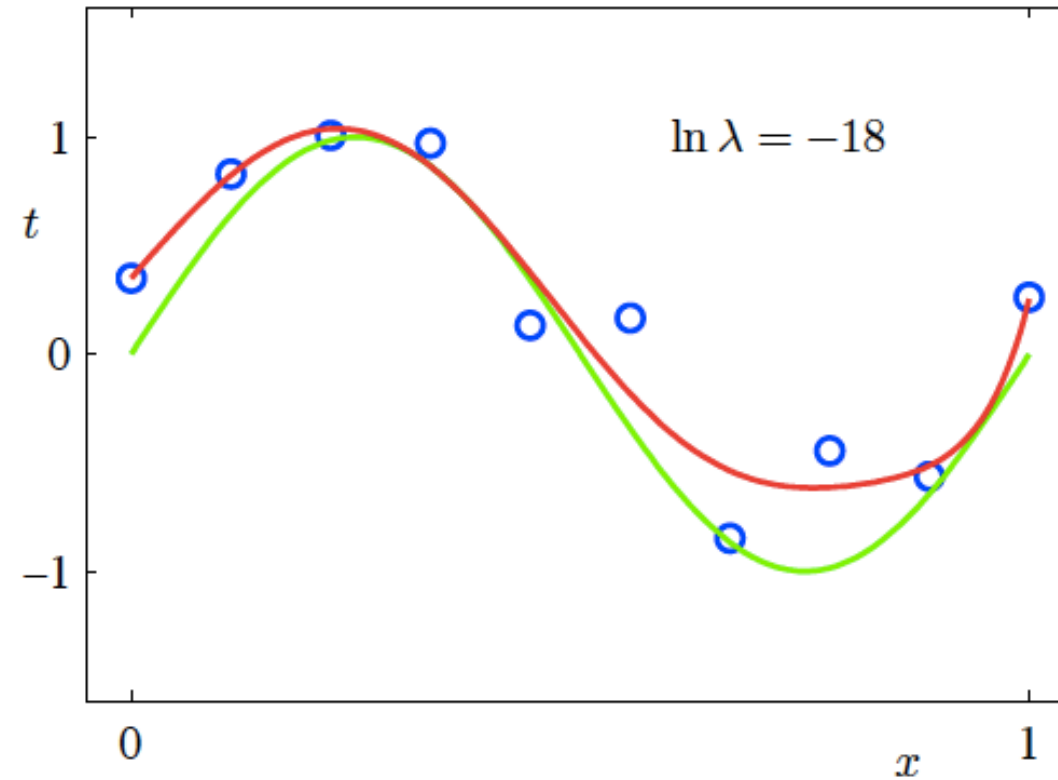
$$\|\mathbf{w}\|^2 = w_0^2 + w_1^2 + \cdots + w_M^2$$

- λ governs the relative importance of the regularisation term compared with the sum-of-squares error term
 - $\ln \lambda = -\infty, \ln \lambda = -18, \ln \lambda = 0$



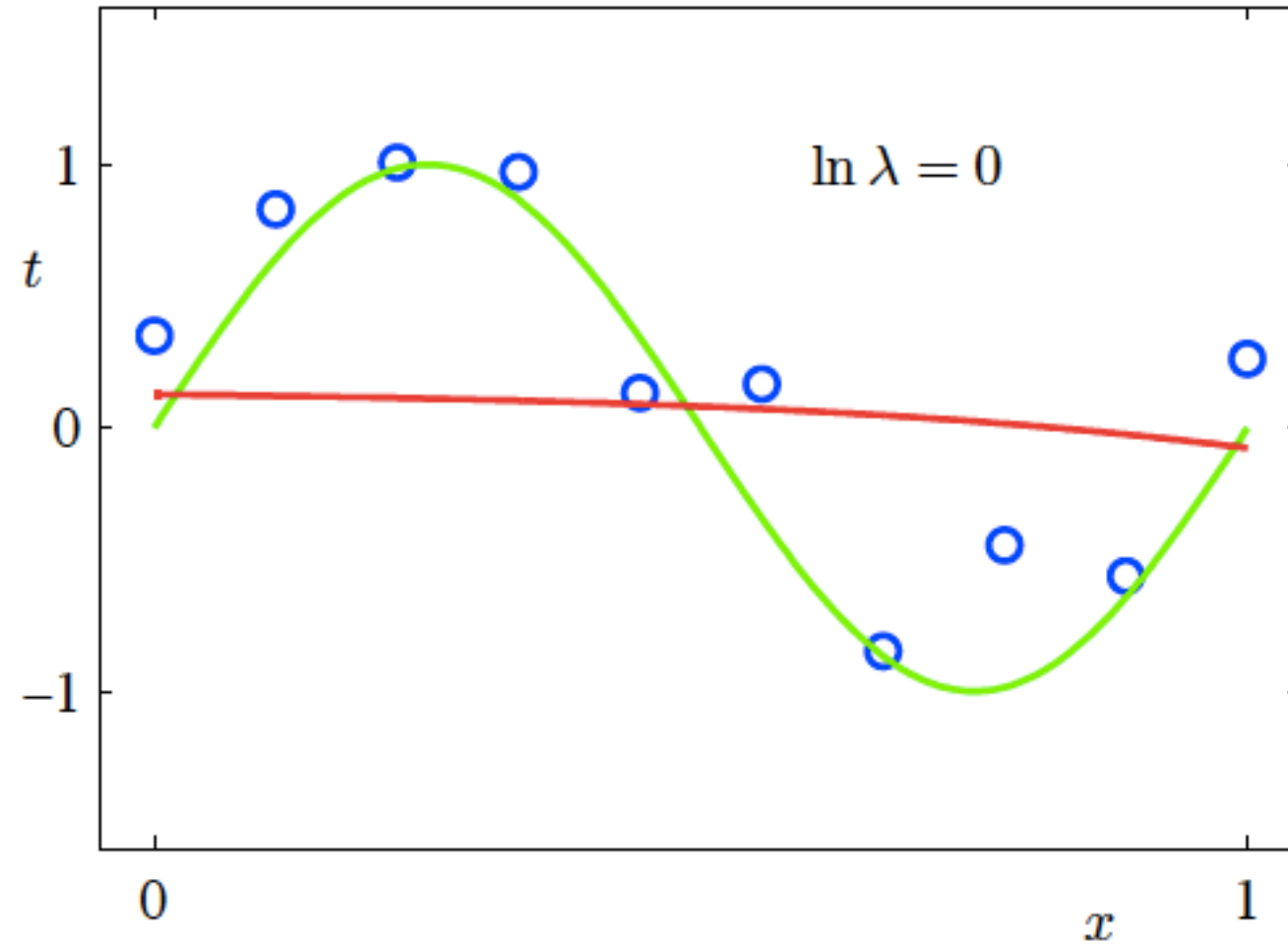
Effect of Regularisation Strength

- $M = 9$



Fitted model ($\ln \lambda = -18$, weak regularisation) (Source: Bishop and Nasrabadi (2006), Figure. 1.7)

Effect of Regularisation Strength (continued)

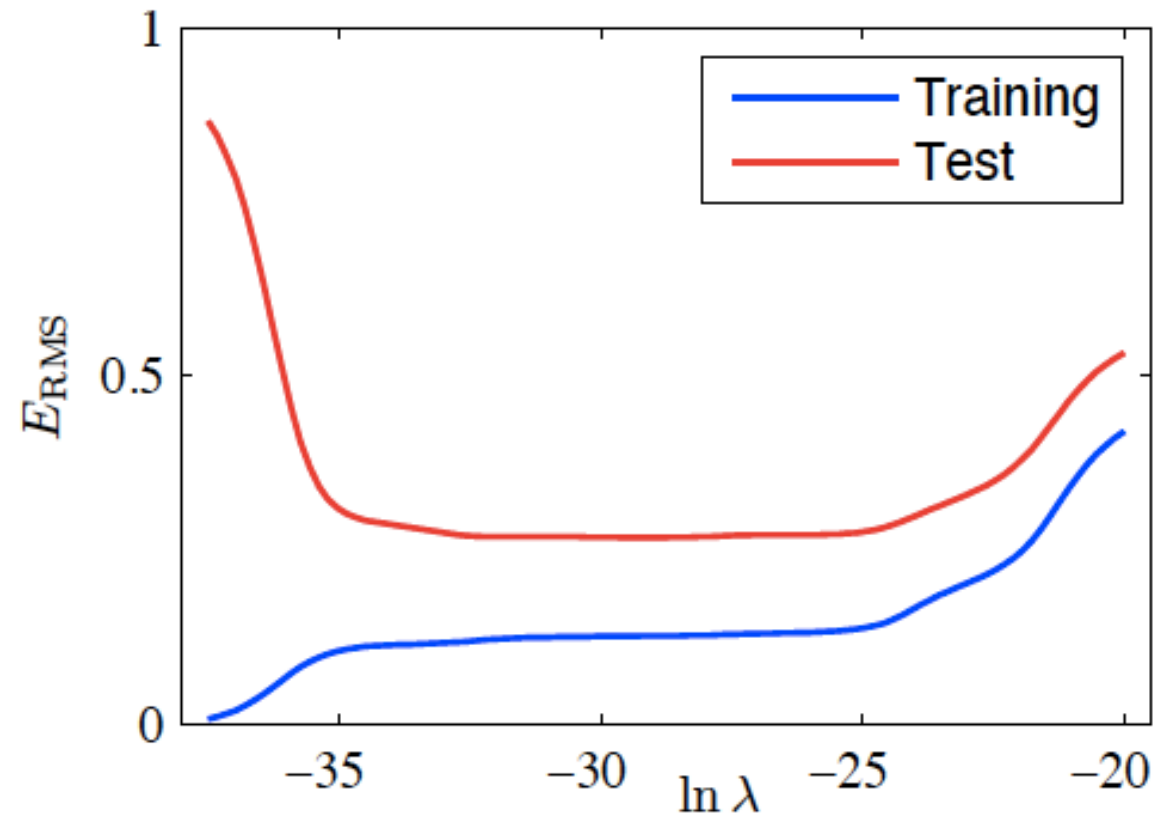


Fitted model ($\ln \lambda = 0$, strong regularisation) (Source: Bishop and Nasrabadi (2006), Figure. 1.7)



RMS Error vs Regularisation

- $M = 9$



Training and test RMS error versus $\ln \lambda$ (Source: Bishop and Nasrabadi (2006), Figure. 1.8)



Shrinkage Techniques for Linear Models



Introduction

Let us begin by reviewing the simplest possible model:

- Sample size: n
- One observation $(\mathbf{x}, y)$:
 - Input feature vector $\mathbf{x} = [x_0, x_1, x_2, \dots, x_p]^T \in \mathbb{R}^{p+1}$, where $x_0 \equiv 1$
 - Output scalar y
- The model $\bar{y} = \boldsymbol{\beta}^T \mathbf{x}$
 - $\boldsymbol{\beta}$ is the parameter vector
- Training:
 - We typically use the mean squared error (MSE):

$$\text{Err}(\boldsymbol{\beta}) = \frac{1}{n} \sum_{i=1}^n (y_i - \boldsymbol{\beta}^T \mathbf{x}_i)^2$$

- The estimator is obtained by: $\hat{\boldsymbol{\beta}} = \underset{\boldsymbol{\beta}}{\text{argmin}} \text{Err}(\boldsymbol{\beta})$



Computation

- Reformulate the loss,
 - Stack the input vectors row by row:

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^T \\ \mathbf{x}_2^T \\ \vdots \\ \mathbf{x}_n^T \end{bmatrix} \in \mathbb{R}^{n \times (p+1)}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} \in \mathbb{R}^n$$

- Rewrite the error: $\text{Err}(\boldsymbol{\beta}) = (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^T(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$
- Closed-form solution (normal equations): $\hat{\boldsymbol{\beta}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$
 - $\mathbf{X}^T \mathbf{X}$ is called the Gram matrix
 - In practice, matrix decomposition methods are used for numerical stability
- In R,

```
1 result = lm(y~X) # append another intercept
2 result = lm(y~X+0) # "+0" implies no intercept
```

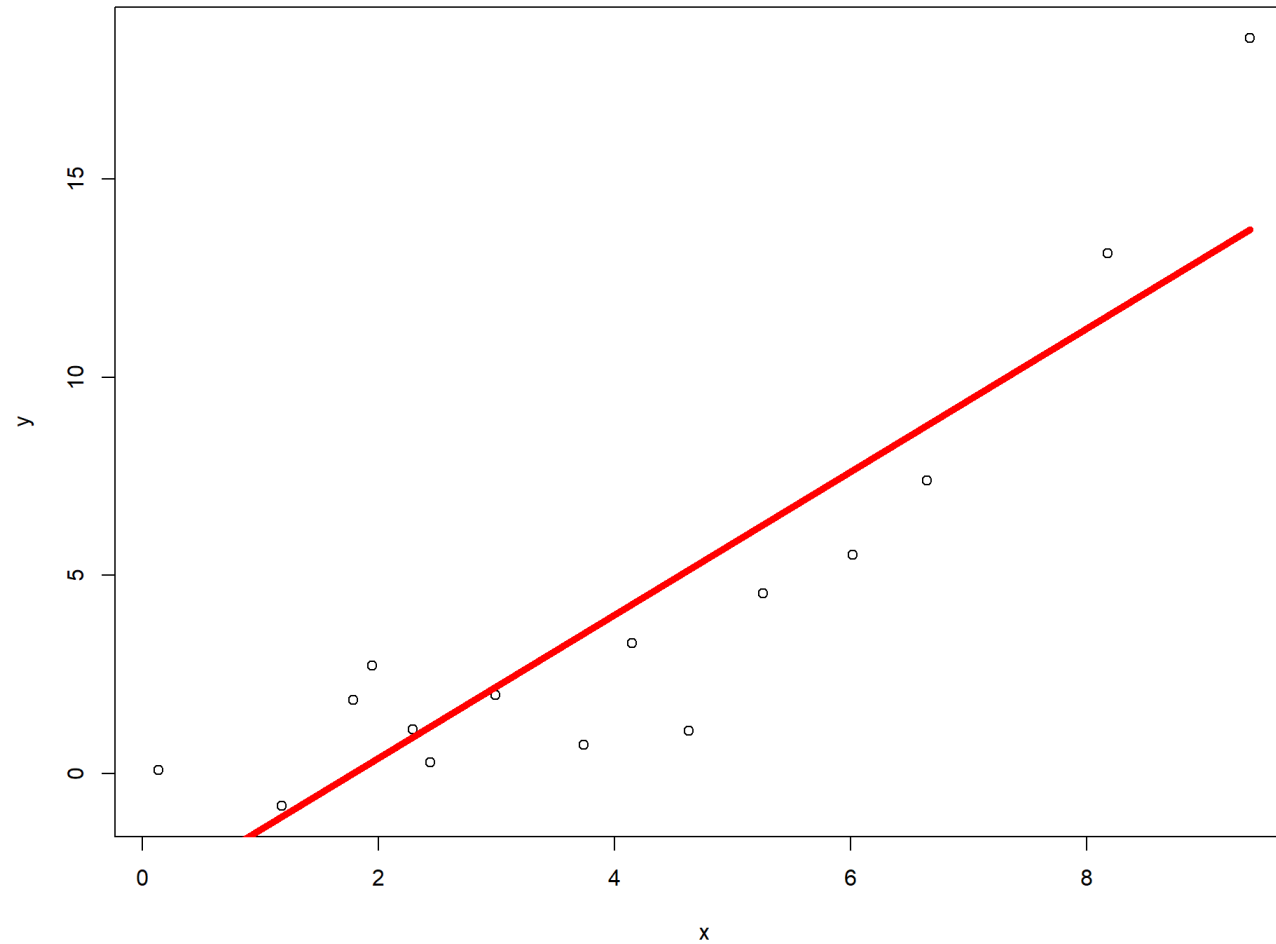


Pros and Cons

- Pros
 - Simple (easy to understand, often generalises well)
 - Fast
 - Well-developed diagnostic tools and theoretical foundations
- Cons
 - May be too simple for complex relationships
 - Training can become unstable when collinearity is present



Too Simple: Linear Model Misspecification



Feature extension

To extend linear regression, a simple approach is to expand the features using basis functions. For example:

1. Polynomial: $x \mapsto (x, x^2, x^3)$

2. Gaussian: $x \mapsto \left(x, e^{-\frac{(x-\mu_1)^2}{2\sigma_1^2}}, e^{-\frac{(x-\mu_2)^2}{2\sigma_2^2}} \right)$

3. Trigonometric: $x \mapsto (x, \sin x, \cos x)$

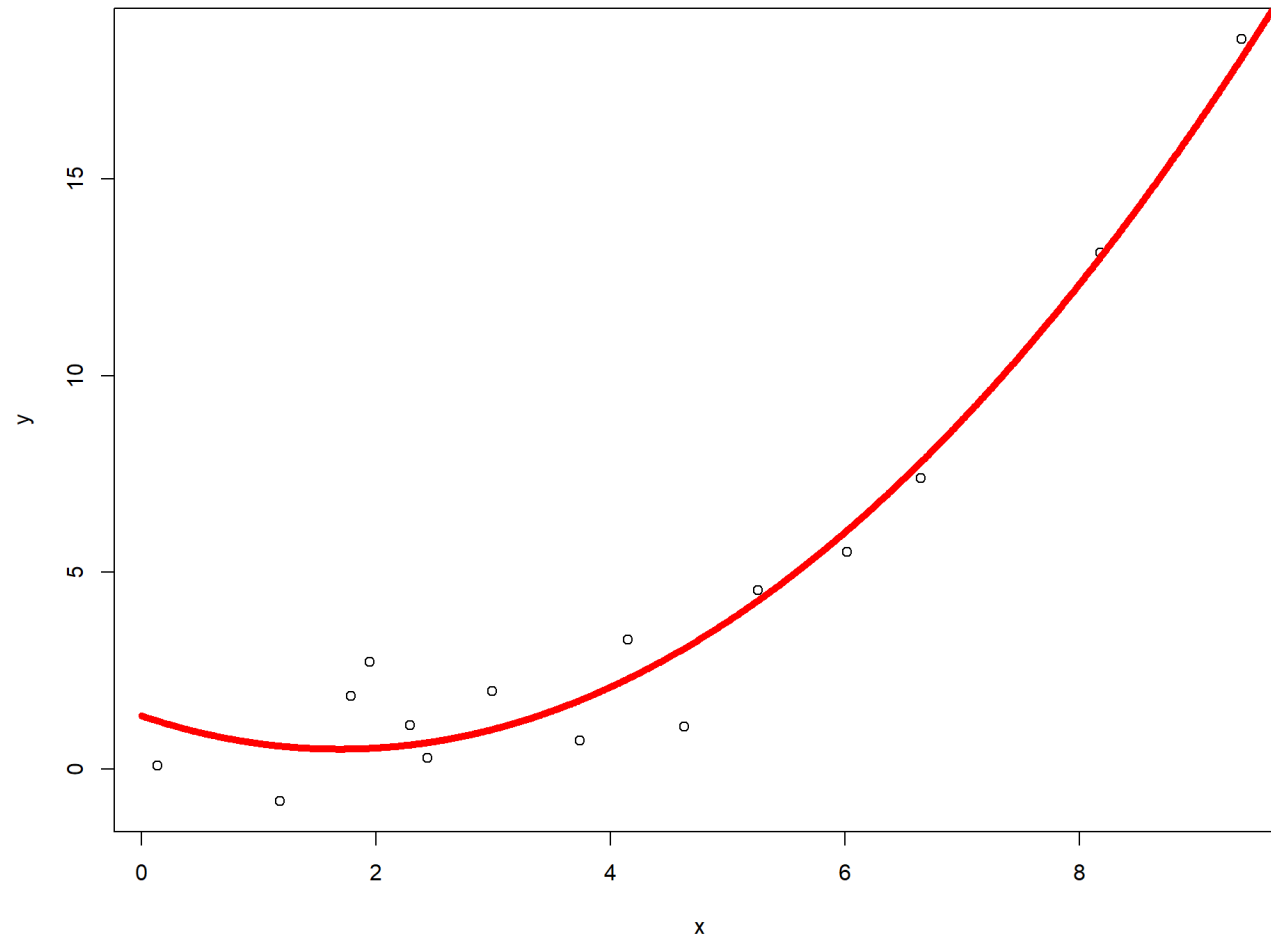
- Linear basis function model:

$$f(x, \boldsymbol{\beta}) = \beta_0 + \sum_{j=1}^{M-1} \beta_j \phi_j(x)$$

- $\phi_j(x)$ are known as *basis functions*

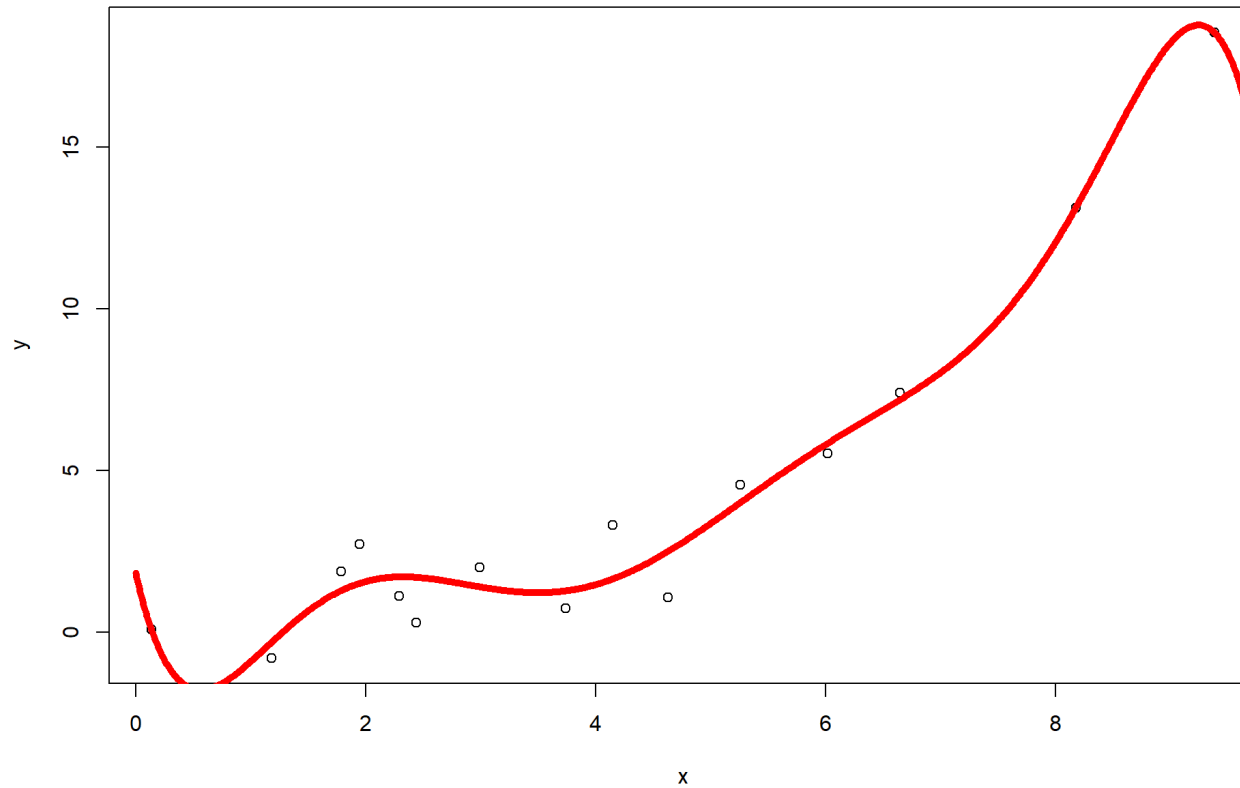


Using (x, x^2)



Overfitting

Sometimes, when the feature dimension is large relative to the number of observations, the model can *overfit*.



Shrinkage Techniques on Linear Basis Functions

- The regularised error is:

$$\tilde{\text{Err}}(\boldsymbol{\beta}) = \sum_{n=1}^N [f(\phi(x_n), \boldsymbol{\beta}) - y_n]^2 + \lambda \|\boldsymbol{\beta}\|^q$$

- $q = 1$: lasso (L1 regularisation)
- $q = 2$: ridge (L2 regularisation)



Bayesian Linear Regression

- Bayes' theorem:

$$\text{posterior} = \frac{\text{likelihood} \times \text{prior}}{\text{normalisation}} \quad p(\boldsymbol{\beta} \mid \mathbf{Y}, \mathbf{X}) = \frac{p(\mathbf{Y} \mid \mathbf{X}, \boldsymbol{\beta}) p(\boldsymbol{\beta})}{p(\mathbf{Y})}$$

- Aim: determine the posterior distribution of the parameters
- When the regression model has errors that follow a normal distribution, and if a particular form of prior distribution is assumed, explicit results are available for the posterior distributions of the model's parameters.



Bayesian Interpretation of L1 and L2 Regularisers

- Assume a linear model with Gaussian errors
- Assume the coefficient vector β has a prior distribution $p(\beta)$
- Assume

$$p(\beta) = \prod_{j=1}^p g(\beta_j)$$

for some density function g

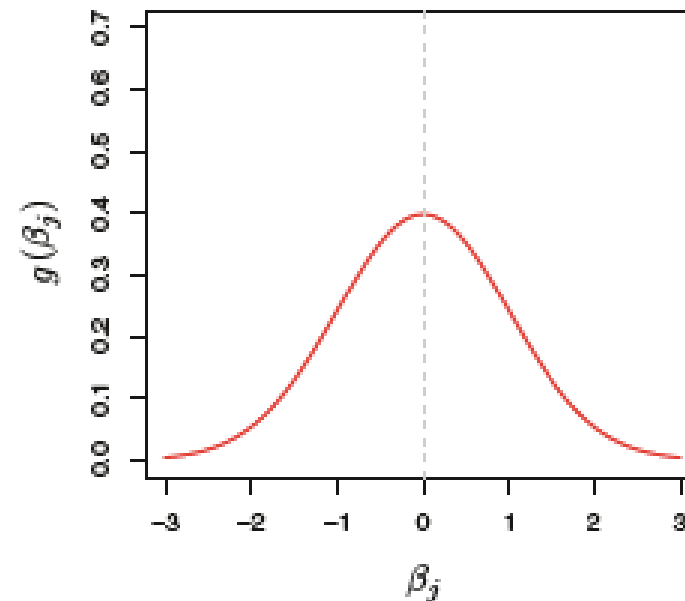
- The likelihood of the data is $p(\mathbf{Y} \mid \mathbf{X}, \beta)$
- The posterior distribution:

$$p(\beta \mid \mathbf{Y}, \mathbf{X}) \propto p(\mathbf{Y} \mid \mathbf{X}, \beta) p(\beta)$$



Bayesian Interpretation of L_2 (Ridge) Penalty

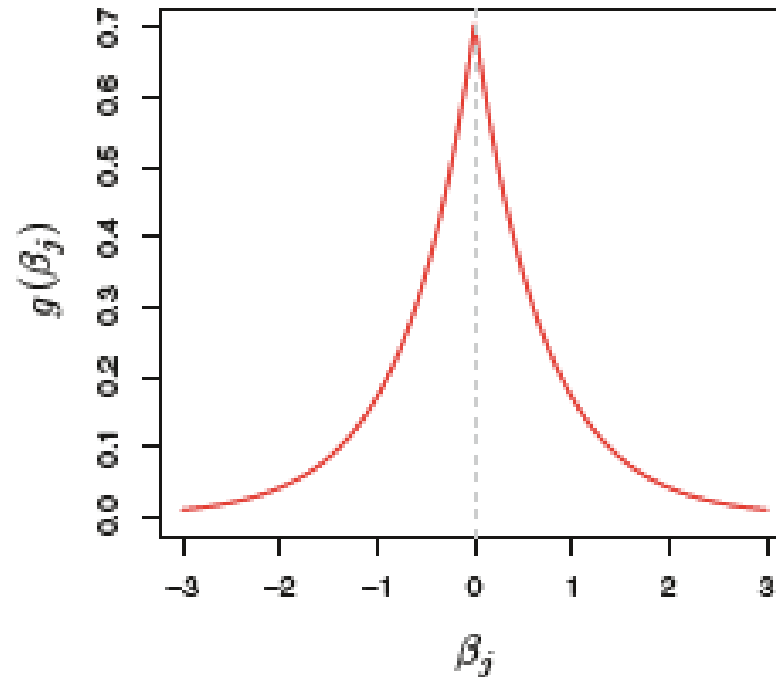
- The prior distribution g is Gaussian with mean zero and standard deviation that depends on λ
- The posterior mode (the most likely value) of β is the ridge regression solution



Gaussian prior for ridge regression (Source: *An Introduction to Statistical Learning* (James et al. 2013), Figure. 6.11)

Bayesian Interpretation of L_1 (Lasso) Penalty

- The prior distribution g is a double-exponential (Laplace) distribution with mean zero and scale parameter that depends on λ
- The posterior mode of β is the lasso solution



Laplace prior for lasso regression (Source: *An Introduction to Statistical Learning* (James et al. 2013), Figure. 6.11)



The Limitations of LASSO

- If the number of predictors (p) exceeds the number of samples (n), the lasso selects at most n variables
 - The number of selected variables is bounded by the sample size
 - Example: Leukemia Data, Golub et al. Science 1999
 - 38 training samples, 34 test samples, and $p = 7129$ genes
- Grouped variables: the lasso fails to perform grouped selection
 - It tends to select one variable from a group and ignore the others
 - Example: For genes that share the same biological “pathway”, the correlations among them can be high. We think of these genes as forming a group

Source: adapted from *Zou and Hastie (2004)*



Elastic Net

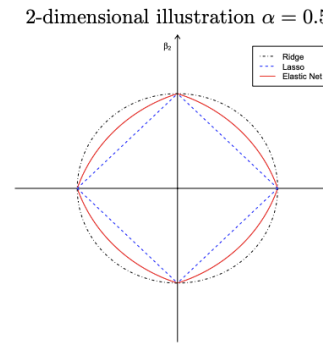
$$\hat{\boldsymbol{\beta}} = \arg \min_{\boldsymbol{\beta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 + \lambda_2 \|\boldsymbol{\beta}\|^2 + \lambda_1 \|\boldsymbol{\beta}\|_1$$

- The L_1 penalty induces sparsity
- The L_2 penalty:
 - Removes the limitation on the number of selected variables;
 - Encourages the grouping effect
 - Stabilizes the l_1 regularization path.

Source: adapted from *Zou and Hastie (2005)*



Geometry of the Elastic Net



Geometry of ridge, lasso, and elastic net penalties (Source: Zou and Hastie (2004))

The elastic net penalty

$$J(\boldsymbol{\beta}) = \alpha \|\boldsymbol{\beta}\|^2 + (1 - \alpha) \|\boldsymbol{\beta}\|_1$$

where

$$\alpha = \frac{\lambda_2}{\lambda_2 + \lambda_1}$$

- Singularities at the vertexes (necessary for sparsity) - Strict convex edges; the strength of convexity varies with α (grouping effect)

Shrinkage Techniques on Generalised Linear Models (GLM)

Regularised GLM

- Linear regression
- Logistic regression
 - Binomial
 - Multinomial
- Poisson regression
- Cox proportional hazards model



Shrinkage Techniques on Other Models



Other Shrinkage Applications

- Sparse PCA
 - Zou, Hastie, and Tibshirani (2006)
- Regularised Clustering
 - Sun, Wang, and Fang (2012)
- Regularised Random Forest
 - Deng and Runger (2013)
- Regularised Neural Network
 - Mc Loone and Irwin (2001)



Using R



Regularised Regression in R

- Ridge, lasso, and elastic net:
 - `glmnet`, `cv.glmnet` (package: `glmnet`)
 - See also: [Introduction to glmnet](#)
- Ridge regression (alternative):
 - `lm.ridge` (package: `MASS`)
- Lasso (alternative):
 - `lars`, `cv.lars` (package: `lars`)
- Penalised regression with cross-validation (alternative):
 - `penalized` (package: `penalized`)



Feature Selection



Considerations for Feature Selection

- Domain knowledge
- Prediction accuracy vs interpretability
- Nuisance variables
- Information leakage
 - e.g. performing feature selection using the full dataset before cross-validation
- Variable interactions
- Highly correlated variables (multicollinearity)
 - Linear models: unstable or highly variable estimates
 - Random forests: mask interactions between features
- Simpler models are often more interpretable



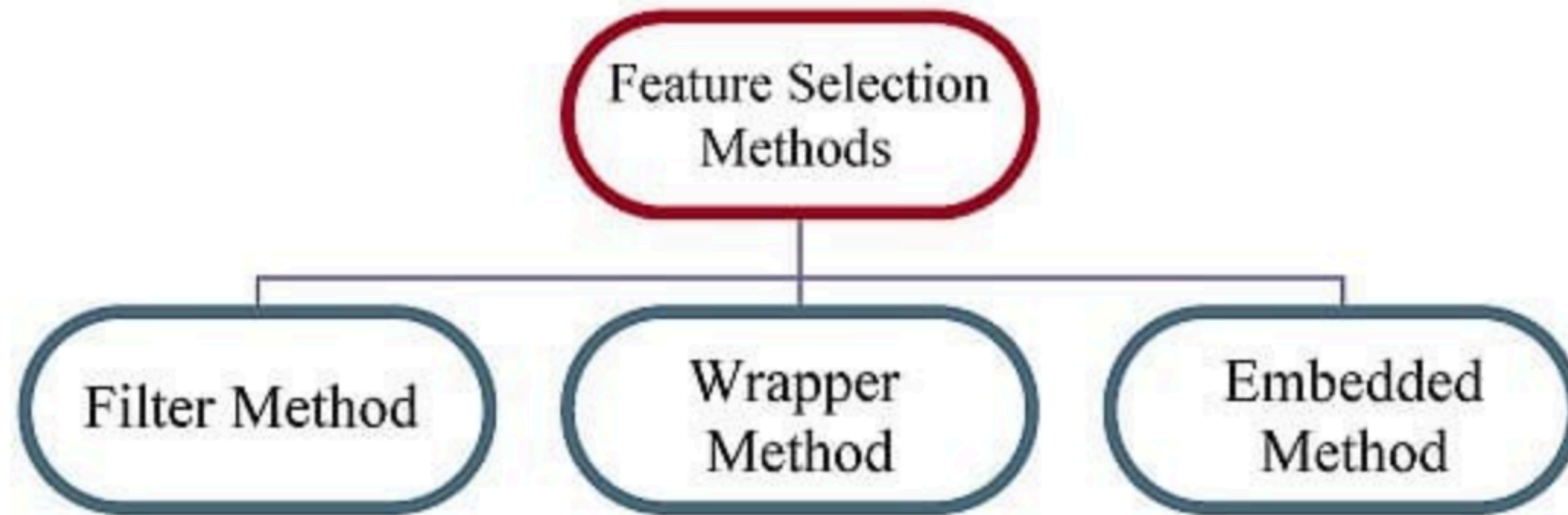
Reasons for Feature Selection

“Sometimes less is better.”

- It can enable faster training
- It reduces model complexity and improves interpretability
- It can improve predictive accuracy if an appropriate subset is selected
- It helps reduce overfitting



Feature Selection Methods



Overview of feature selection methods (Source: Analytics Vidhya (2016), adapted from online sources)

Filter Methods



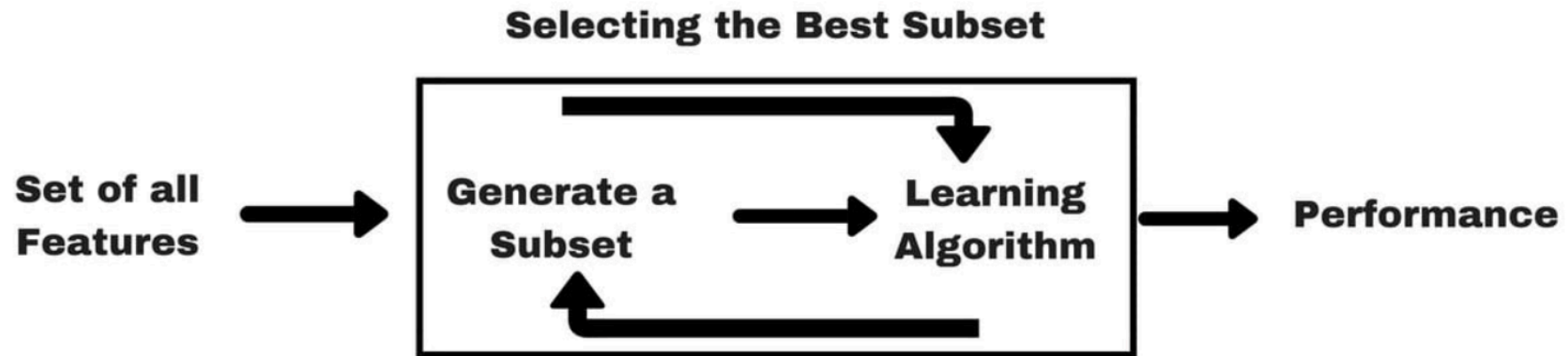
Filter methods (Source: Analytics Vidhya (2016))

- Used as a data preprocessing step
- Features are selected based on statistical measures of their relationship with the target (response) variable
- Filter methods do not address multicollinearity.

Feature\Response	Continuous	Categorical
Continuous	Pearson's Correlation	LDA
Categorical	ANOVA	Chi-square test



Wrapper Methods



Wrapper methods (Source: Analytics Vidhya (2016))

- Identify subsets of features by training and evaluating a model
- This is a search problem and can be computationally expensive.

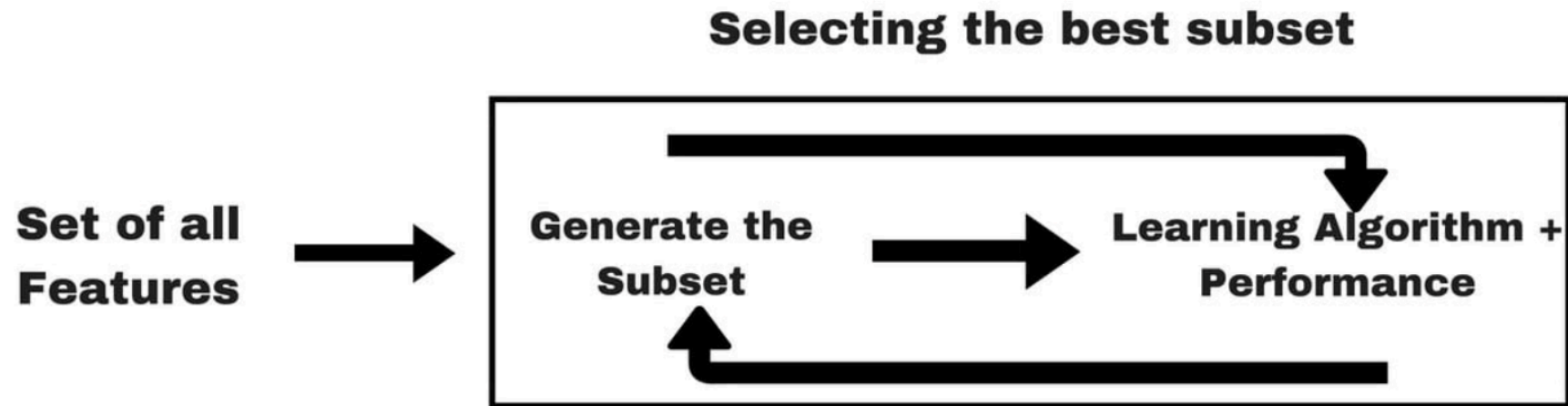
Wrapper Methods Examples

- Best subset selection (2^p models)
- Stepwise selection
 - Forward stepwise selection ($1 + p(p + 1)/2$ models)
 - Backward stepwise elimination ($1 + p(p + 1)/2$ models)
 - Hybrid approaches

Review: *An Introduction to Statistical Learning* (James et al. 2013), Chapters 6.1, 6.5



Embedded Methods



Embedded methods (Source: Analytics Vidhya (2016))

- Feature selection is built into the model training process
- Example: lasso regression

Dimension Reduction

- Transform predictors to reduce the dimensionality of the problem
- Purposes?
- Methods:
 - PCA
 - PCR
 - PLS
 - Compare their advantages and disadvantages

Review: *An Introduction to Statistical Learning* (James et al. 2013), Chapters 6.3, 10.2



References

- Analytics Vidhya. 2016. "Introduction to Feature Selection Methods with an Example."
<https://www.analyticsvidhya.com/blog/2016/12/introduction-to-feature-selection-methods-with-an-example-or-how-to-select-the-right-variables/>.
- Bishop, Christopher M, and Nasser M Nasrabadi. 2006. *Pattern Recognition and Machine Learning*. Vol. 4. 4. Springer.
- Cochrane, Courtney. 2018. "Time Series Nested Cross-Validation." May 2018. <https://medium.com/data-science/time-series-nested-cross-validation-76adba623eb9>.
- Deng, Houtao, and George Runger. 2013. "Gene Selection with Guided Regularized Random Forest." *Pattern Recognition* 46 (12): 3483–89.
- James, Gareth, Daniela Witten, Trevor Hastie, Robert Tibshirani, et al. 2013. *An Introduction to Statistical Learning: With Applications in r*. Vol. 103. Springer.
- Mc Loone, Seán, and George Irwin. 2001. "Improving Neural Network Training Solutions Using Regularisation." *Neurocomputing* 37 (1-4): 71–90.
- Sun, Wei, Junhui Wang, and Yixin Fang. 2012. "Regularized k-Means Clustering of High-Dimensional Data and Its Asymptotic Consistency."
- Wikimedia Commons contributors. 2016. "K-Fold Cross Validation (English)." 2016.
https://commons.wikimedia.org/wiki/File:K-fold_cross_validation_EN.jpg.
- Zou, Hui, and Trevor Hastie. 2004. "Regularization and Variable Selection via the Elastic Net." 2004.
https://web.stanford.edu/~hastie/TALKS/enet_talk.pdf.
- — —. 2005. "Regularization and Variable Selection via the Elastic Net." *Journal of the Royal Statistical Society Series B: Statistical Methodology* 67 (2): 301–20.
- Zou, Hui, Trevor Hastie, and Robert Tibshirani. 2006. "Sparse Principal Component Analysis." *Journal of Computational and Graphical Statistics* 15 (2): 265–86.

